Faculty of Computers and Artificial Intelligence

Course: Event-Driven Programming

Project-To-Project(P2P)

Windows Forms Application Launcher Hub & Sign Language Dictionary Module

Prepared By (Project Team)

Philopateer Waheed	ID: 42410086
Shimaa Salah	ID: 42410338
Noran Ahmed	ID: 42410001
Yousef Mohamed	ID: 42410515
Kerolos Nady	ID: 42410542
Shrouk Khalaf	ID: 42410575

Presented To (Instruction Team)

Dr. Ahmed Seif	Course Professor
Eng. Ahmed Karem	Teaching Assistant
Eng. Ahmed Hamdy	Teaching Assistant

Final Term Project Report

Academic Year: 2025 / 2026

Project Report

Technical Documentation & Architecture Review

1. Executive Summary

The **Event_Project** is a professional Windows Forms desktop application developed in C# utilizing the advanced features of the .NET 10 framework. The system is engineered to serve a strategic dual purpose within a single, unified environment:

1. **Project Hub & Launcher:** It functions as a centralized dashboard allowing users to dynamically register external executable (.exe) files into a persistent database and launch them directly from the primary application interface.
2. **Sign Language Dictionary:** It hosts an internal, highly responsive application module that translates text-based queries into sign language visual demonstrations via animated GIFs.

To provide a zero-configuration installation footprint and an optimal user setup experience, the application leverages SQL Server LocalDB. It automatically detects, generates, and attaches an isolated .mdf database file upon application startup, completely removing external dependencies or manual server configurations.

2. Architecture & Design Specifications

The foundational infrastructure of the system adheres to traditional desktop design architectures, modernized for the .NET ecosystem:

- **Target Framework:** .NET 10 Windows Forms (WinForms) for a native, high-performance user interface execution.
- **Database Engine:** Microsoft SQL Server LocalDB (MSSQLLocalDB). The application isolates state data by auto-generating the physical file EventProjectDB.mdf securely within the user's local MyDocuments directory.
- **Data Access Pattern:** Low-level, high-performance raw ADO.NET using the modern Microsoft.Data.SqlClient provider. System interactions rely strictly on low-overhead SqlConnection management and parameterized SqlCommand executions.

3. Database Schema Blueprint

The relational schema is lightweight and optimized for rapid lookups. The schemas are verified and structurally generated at runtime by an internal database initialization helper.

Table 1: Projects Table

Maintains references and metadata for registered external executables integrated into the launcher hub.

Column Name	Data Type	Key / Constraints	Functional Description
Id	INT	Primary Key, Identity	Unique system identifier for the registered executable.
Name	NVARCHAR(100)	Not Null	The user-friendly display name of the external application within the hub.
ExePath	NVARCHAR(MAX)	Not Null	The absolute, validated physical filepath to the application executable file.

Table 2: Signs Table

Acts as the backend storage mapping text values to their corresponding graphical sign language representations.

Column Name	Data Type	Key / Constraints	Functional Description
Id	INT	Primary Key, Identity	Unique system

Column Name	Data Type	Key / Constraints	Functional Description
			identifier for the data dictionary entry.
Answer	NVARCHAR(100)	Not Null, Index optimized	The precise textual word or phrase entry matching user search queries.
GifPath	NVARCHAR(MAX)	Not Null	The relative, portable application path to the GIF file illustrating the sign.

4. Core Components & Modular File Breakdown

The codebase is logically split into targeted separation of concerns across multiple core functional classes and view structures:

Component / File	Architectural Role	Primary Responsibilities & Logic Handles
Program.cs	Application Entry Point	• Invokes DatabaseHelper.EnsureDatabaseCreated() to bootstrap state schemas on cold starts. • Intercepts the custom --test command-line argument to safely bypass the GUI and run console-based validations. • Initializes and executes Form1 within the main

Component / File	Architectural Role	Primary Responsibilities & Logic Handles
		thread.
DatabaseHelper.cs	Centralized Data Management	• Orchestrates raw connection sequences via temporary master DB targets to safely construct local storage profiles. • Exposes uniform ADO.NET Connection Strings across internal entities. • Maintains functional data routines including GetAllWords() and lookup optimizations via GetGif().
DatabaseTester.cs	Self-Testing Engine	• Executes pre-flight pipeline checks to catch environmental file configuration errors early. • Performs programmatic database validation through mock record orchestration. • Assures data integrity across standard Create and Read (CR) database transactions.
Form1.cs	The Launcher Hub View	• Renders dynamically populated collections of custom registered external tool components. • Maintains a safe, static bridge to run the internal dictionary module (INTERNAL_FORM2).

Component / File	Architectural Role	Primary Responsibilities & Logic Handles
		• Integrates native OpenFileDialog controls to parse and append clean target executables smoothly.
Form5.cs	Sign Language Interface	• Populates standard word indexes asynchronously from database queries. • Maps selected user text criteria to target GIF structures and pushes updates to a PictureBox (pbGif). • Exposes administrator/management pipelines to drop or refresh entries from the active dataset.

5. Architectural Strengths

- **Self-Healing Storage Pipeline:** The automated implementation of EnsureDatabaseCreated gracefully handles fresh bare-metal setups. This eliminates the necessity for engineers or end-users to manually establish schemas or run tedious out-of-band .sql preparation configurations.
- **Integrated Diagnostics Core:** Standardizing runtime parameters via the --test CLI mode decoupled from visual components allows for seamless integration into modern automated continuous integration (CI/CD) environments or rapid deployment diagnostics.
- **Extensible Application Topology:** Building the master application layout around a centralized launcher form gives the architectural platform a highly structured path forward if additional modular sub-systems are introduced.

6. Observations & Strategic Recommendations

1. Object-Relational Mapping Transition (EF Core / Dapper)

While the existing database layer utilizes parameterized queries to mitigate security vulnerabilities like SQL Injection, relying entirely on raw ADO.NET presents long-term maintainability concerns. Transitioning data operations to a lightweight mapper like *Dapper* or a robust ecosystem like *Entity Framework Core* would radically minimize repetitive boilerplate code, provide cleaner strongly typed object mapping, and simplify model handling throughout the application.

2. Form Navigation & Decoupling

The current architecture binds INTERNAL_FORM2 as a hardcoded solution. Designing and implementing a concrete application-routing layout or establishing an abstract interface interface (such as IPlayableProject) would allow the hub to handle embedded native WinForms modules and independent external binaries identically, maximizing systemic cohesion.

3. Data Access Isolation & Connection Lifecycles

Connection handling configurations are currently distributed globally across individual forms. Adopting the *Repository Pattern* in conjunction with basic *Dependency Injection (DI)* principles to safely supply a shared, scoped DatabaseHelper structure across views would guarantee connection lifecycles are consistently opened, monitored, and safely disposed of.

4. Enterprise Exception Strategy & Logging

Currently, functional exception anomalies within Form1 and Form5 output to localized developer debug logs or simple visual alert popups. It is highly recommended to formalize error handling with a centralized platform engine such as *Serilog* or *NLog* to ensure runtime faults are recorded uniformly and securely for diagnostic evaluation.